Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Evie Dryden	egdryden@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

A goal or problem the project wants to solve is that it is hard to organize and for someone to want to look at boring chemistry textbooks. The website will hold, take new books, let you read basic information about them, and give you a score based on how much you've read in a creative chemistry way by adding new topics to read and give recommendations for topics.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

There are similar tools like this such as MyChemLab AI and ChemVerse Virtual Chemistry Lab. Both of these take text and let the user make experiments. This is also similar to

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

The purpose of this site would be to store books and let the person read the title, difficulty, and log if they have read it. It also allows a competitive chemistry pun-filled ranking of how

many chemistry textbooks you've read. I want this to be used by a person to make their textbook purchasing and reading more interactive.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The program will collect and process book information using dataclasses and lists to filter each list to organize books into read and unread categories, then generate personalized recommendations for topics from branch conditions and rankings based on the user's reading history.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The textbooks name, page count, topic, read or not, difficulty, and author.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The computer will manipulate the data by filtering out read or not read and compare the topic against a database of topic recommendations.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will get a score of what books they have read and a recommendation back for topic to read.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Textbook, Rank, Recommendation

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

I need the textbook name, author, page count, topic, read or not, and difficulty. Details I will ignore are textbook brand, publication date, and cost.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

The hardest part is fitting my idea into all the requirements of the project. As well as understanding the coding, as it is more complex than anything we have done. Drafter is slightly confusing coming from an intro Computer Science student.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will use an LLM to help code the actual project and in order to avoid any mistake I will make sure to run the code. Additionally, I will consult multiple different LLMs and I will check their work. I will make assert equals to help with this.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

My goal is to impress my chemist professor mom. I hope that she will be able to enjoy it.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a simple web tool to help students organize their chemistry textbooks, get a rank of the difficulty of the books they have read, and get new recommendations based on the topics they have read. What are 3 common problems students face when reading textbooks, and what are 3 simple features a basic tool should have?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The AI's best suggestions was to implement the individual chapters completion marker and make a prerequisite based book recommendation. There are existing apps like Goodreads and Notion.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

A global difficulty meter is too difficult to for an introductory programming project because I am unsure how to implant crowdsourced information in drafter, as I have never collected and stored information across multiple users. Please give a simpler alternative.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

It has become more focused to include a prerequisite recommendation as a suggestion. Now my project idea is that the website will hold, take new books, let you read basic information about them, and give you a rank based on how much you've read in a creative chemistry way and give recommendations based on your previous books.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will use google and previous drafter workshops in the class. If I get super stuck I will ask a TA.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will first read the error message then look at why it's failing. If I can not fix it myself I will provide the LLM with the error and ask why it's not working. I will also ask how I can fix the code.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

> "I want to build a simple web tool to help students organize their chemistry textbooks, get a rank of the difficulty of the books they have read, and get new recommendations based on the topics they have read. What are 3 common problems students face when reading textbooks, and what are 3 simple features a basic tool should have?"

Students frequently struggle to absorb massive amounts of academic text efficiently. When designing a tool to help them organize and rank their reading, addressing their core cognitive and logistical friction points is key.

Here are **3 common problems** students face when reading textbooks, followed by **3 simple features** a basic web tool should have to solve them.

3 Common Problems Students Face

- **Information Overload and Density:** Chemistry textbooks are dense, packed with complex formulas, abstract molecular structures, and heavy jargon. Students often experience cognitive fatigue quickly, making it hard to retain what they just read.
- **Passive Reading Habits:** Many students read textbooks passively (skimming paragraphs or highlighting entire pages) rather than actively processing the concepts. This leads to the "illusion of competence," where they feel like they understand the material until they try to solve a problem.
- **Lack of Structured Sequencing:** Students often don't know what order to read topics in. Because chemistry builds heavily on foundational concepts (e.g., you need to understand atomic structure before tackling chemical bonding), getting stuck on a hard chapter without mastering the prerequisites causes frustration and dropouts.

---

3 Simple Features a Basic Tool Should Have

To address these pain points while keeping your initial build simple, focus on these three core features:

- **Visual Syllabus Checklist & Tracker:** A dashboard where students can input their textbook's chapters and visually mark them as "Not Started," "In Progress," or "Mastered." This breaks down a massive 1,000-page book into bite-sized, gamified milestones.
- **Crowdsourced "Difficulty Meter" Ranking:** A feature allowing students to rate each chapter or book on a scale of 1–5 after reading it. The tool can aggregate these scores into a public leaderboard, warning future students about notoriously difficult

sections (like Thermodynamics or Orbitals) so they can allocate more study time.
- **Prerequisite-Based Topic Recommender:** A simple recommendation engine that maps keywords or chapters. If a student marks "Introduction to Organic Chemistry" as completed, the tool automatically suggests the next logical step (e.g., "Reaction Mechanisms") or routes them back to foundational topics if they find the current book too hard.

If you want to map out the next steps for your project, let me know:

- What **programming languages or frameworks** you plan to use (e.g., React, plain HTML/JS, Python)?
- Whether you want this to be a **private personal tracker** or a **social platform** where students see each other's ratings?

I can help you sketch out a basic **database schema** or a **wireframe layout** to get you building!

What existing solutions or prior apps did they mention?

In the previous turn, **no existing solutions or prior apps were mentioned**.

The response focused entirely on outlining the abstract conceptual challenges students face (Information Overload, Passive Reading, and Lack of Structured Sequencing) and brainstorming three original feature concepts for your specific web tool (Visual Syllabus Checklist, Crowdsourced Difficulty Meter, and Prerequisite-Based Recommender).

Would you like to see how popular existing platforms like **Notion**, **Goodreads**, or academic trackers manage these kinds of workflows so you can borrow from their designs?

> A global difficulty meter is too difficult to for an introductory programming project because I am unsure how to implant crowdsourced information in drafter, as I have never collected and stored information across multiple users. Please give a simpler alternative.

Instead of crowdsourcing data from multiple users, you can build a **personalized difficulty tracker** that relies entirely on **local user data** stored directly in the browser.

Here are three simple, local alternatives that do not require a database or multi-user setup:

- **Self-Reported Slider:** Let the student rate the difficulty of a chapter themselves (1–5 scale). The app can calculate an average difficulty score just for *their* active books.
- **Time-Based Difficulty Proxy:** Use a simple stopwatch or manual log feature where users track how long it took them to read a chapter. The tool can automatically rank chapters with the highest read-times as the most "difficult."
- **Dynamic Visual Tags:** Allow students to flag a chapter as "Green" (Easy), "Yellow" (Medium), or "Red" (Hard) using simple click buttons, sorting their personal dashboard by these categories.

Technical Implementation for Drafter

To build this without a server, you can store the user's personal rankings using the browser's built-in `localStorage` **API**.

`localStorage` saves data as simple key-value strings directly on the user's computer. The data persists even if they refresh the page, allowing you to build a functioning, personal app with pure frontend code.

javascript

None

```
// Example: Saving a user's difficulty rating for a book
localStorage.setItem("chemistry_book_1", "Hard");

// Example: Retrieving the rating when the page loads
const bookRating = localStorage.getItem("chemistry_book_1");
```

Use code with caution.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.